Phishing URL Detection Using Python

Project Report with Requirements, Explanation, Diagrams, and Visual Analysis

1. Project Overview

Phishing URL detection is a cybersecurity project that identifies malicious links before users open them. The system uses Python to extract features from a URL and applies a machine-learning model to classify it as **phishing** or **legitimate**.

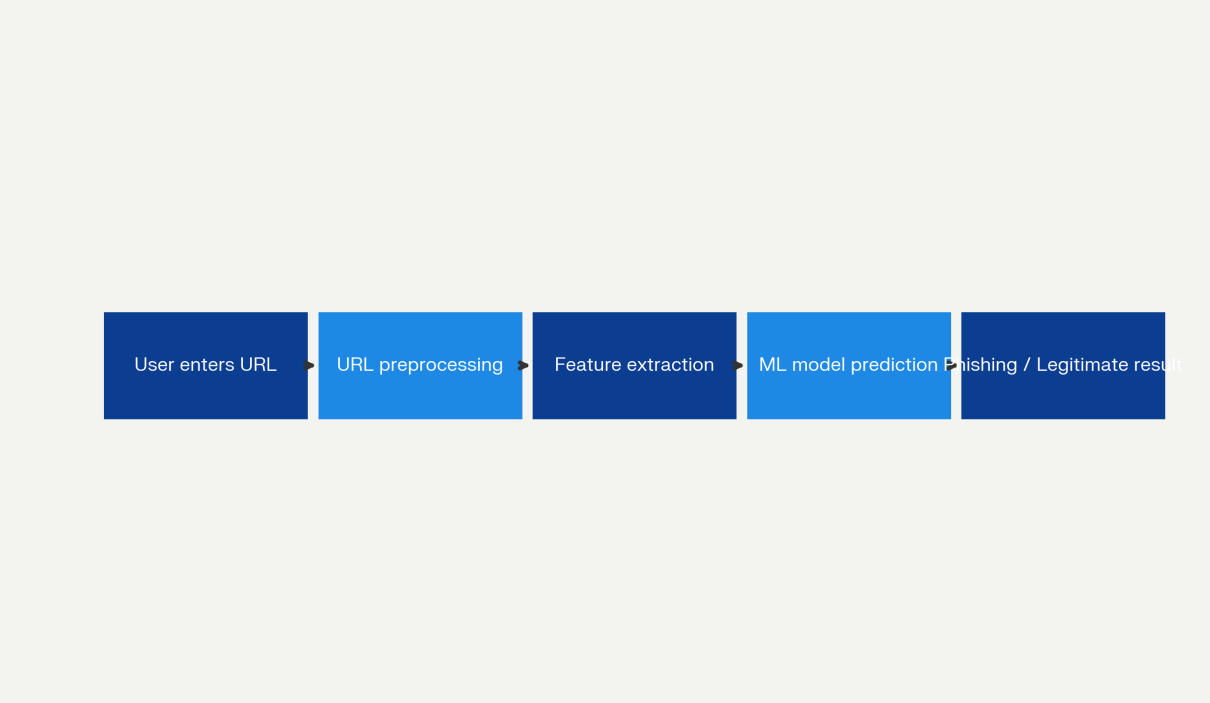


Figure 1: Complete system architecture

2. Requirements File

The **requirements.txt** file lists every Python library needed to run the project. Install all dependencies using:

pip install -r requirements.txt

```
pandas==2.2.3 numpy==2.1.3 scikit-learn==1.6.1 joblib==1.4.2 imbalanced-learn==0.13.0
requests==2.32.3 tldextract==5.1.3 python-whois==0.9.5 dnspython==2.7.0
beautifulsoup4==4.12.3 lxml==5.3.0 matplotlib==3.10.1 seaborn==0.13.2 Pillow==11.1.0
Flask==3.1.0 streamlit==1.41.1 reportlab==4.2.5
```

3. Brief Explanation of Requirements

Library	Purpose in the project
pandas	Reads and processes the URL dataset.
numpy	Performs numerical operations and array handling.
scikit-learn	Trains and evaluates the phishing detection model.
joblib	Saves and loads the trained machine-learning model.
imbalanced-learn	Handles unequal phishing and legitimate URL samples.
requests	Fetches webpage or redirect information when required.
tldextract	Extracts domain, subdomain, and top-level domain details.
python-whois	Checks domain registration and age information.
dnspython	Performs DNS-related checks.
beautifulsoup4	Extracts useful information from webpage HTML.
lxml	Fast HTML and XML parser used with BeautifulSoup.
matplotlib	Creates graphs, charts, and project-report images.
seaborn	Creates advanced statistical visualizations.
Pillow	Handles and processes images.
Flask	Creates a simple web interface for URL checking.
streamlit	Creates an easy interactive dashboard.
reportlab	Generates the final PDF project report.

4. Important URL Features

The model learns from visible URL characteristics. Phishing links often use long addresses, excessive subdomains, IP addresses, misleading keywords, and URL shorteners.

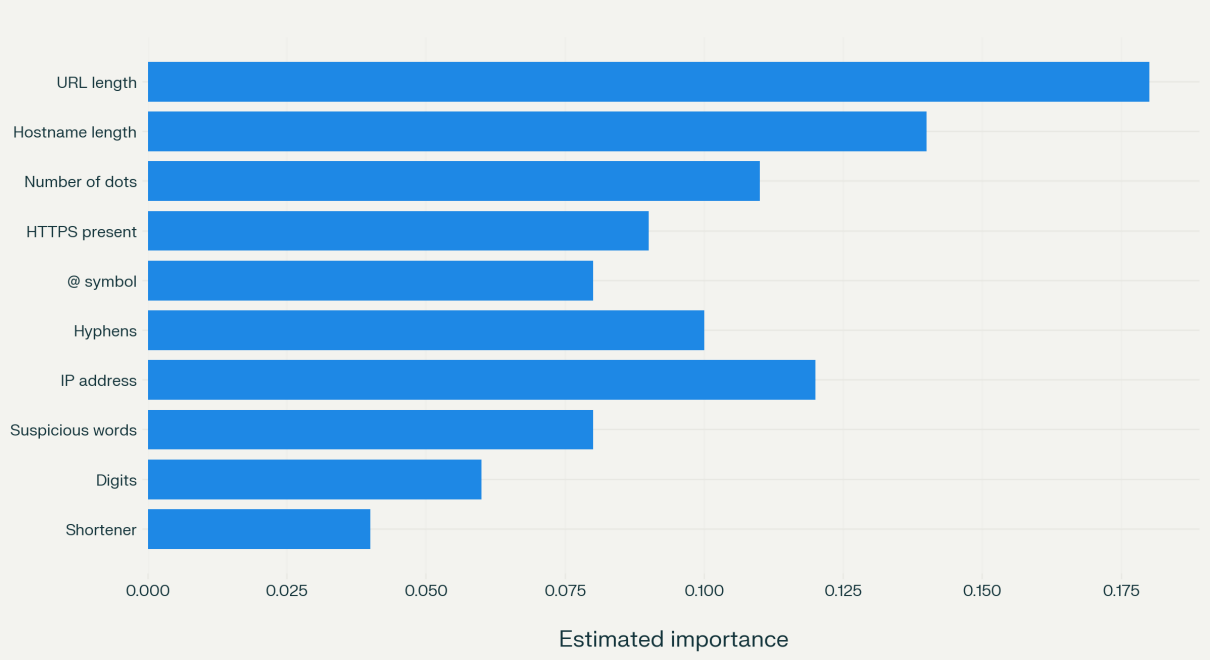


Figure 2: Estimated importance of URL features

5. Warning Signs in Phishing URLs

Users should be careful when a URL contains HTTP instead of HTTPS, an IP address, an @ symbol, too many subdomains, or words such as login, verify, secure, and account.

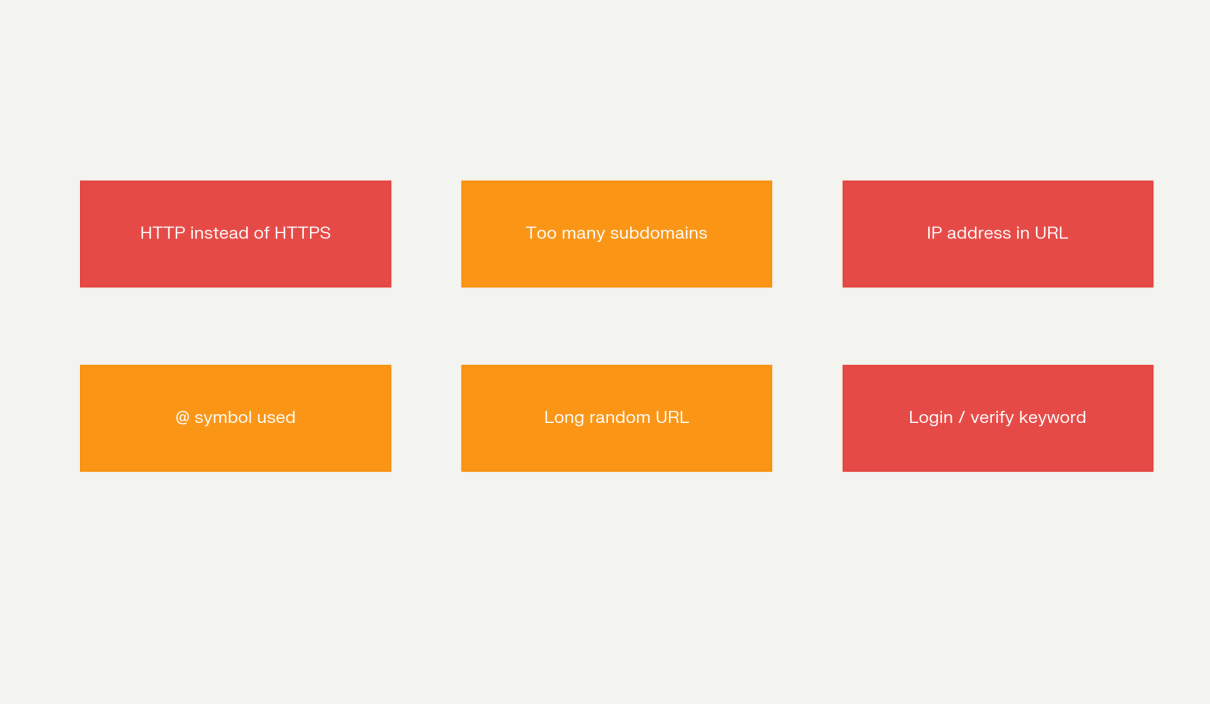


Figure 3: Common phishing URL warning signs

6. Dataset and Model Training

A balanced dataset improves fairness and reduces bias. The dataset should contain both legitimate and phishing URLs, with extracted features and labels: 0 for legitimate and 1 for phishing.

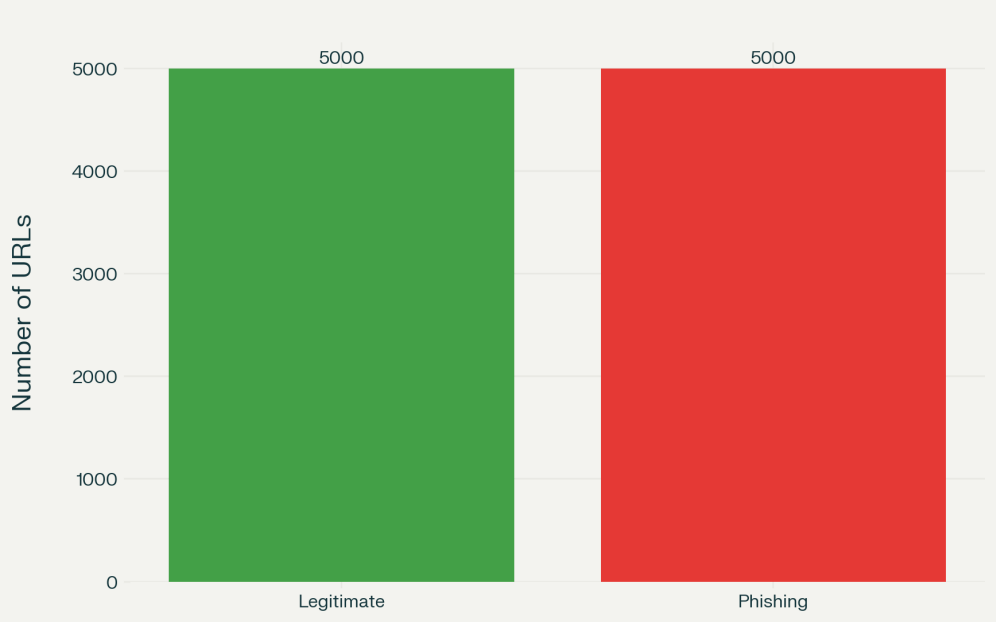


Figure 4: Balanced dataset example

The recommended model is Random Forest because it handles nonlinear URL patterns well and provides feature-importance scores. The dataset is split into training and testing sets, commonly 80% for training and 20% for testing.

```
import pandas as pd from sklearn.model_selection import train_test_split from
sklearn.ensemble import RandomForestClassifier from sklearn.metrics import accuracy_score,
classification_report import joblib data = pd.read_csv("phishing_dataset.csv") X =
data.drop(columns=["label"]) y = data["label"] X_train, X_test, y_train, y_test =
train_test_split( X, y, test_size=0.2, random_state=42, stratify=y ) model =
RandomForestClassifier(n_estimators=300, random_state=42) model.fit(X_train, y_train)
prediction = model.predict(X_test) print("Accuracy:", accuracy_score(y_test, prediction))
print(classification_report(y_test, prediction)) joblib.dump(model,
"phishing_url_model.pkl")
```

7. Model Evaluation

The confusion matrix shows how many URLs were classified correctly and incorrectly. In the example below, the model correctly identifies most legitimate and phishing URLs.

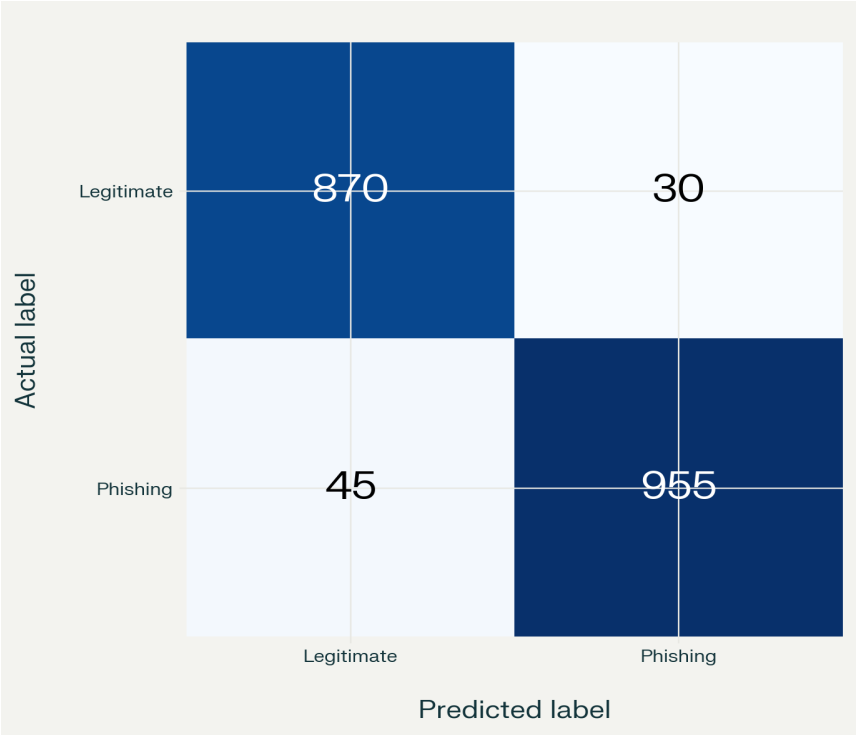


Figure 5: Example confusion matrix

```
Accuracy = (TP + TN) / (TP + TN + FP + FN) Precision = TP / (TP + FP) Recall = TP / (TP + FN)
F1-score = 2 * Precision * Recall / (Precision + Recall)
```

8. Technical Visualization

This visualization represents the continuous analysis of URL patterns, features, and model decisions used in the phishing detection process.

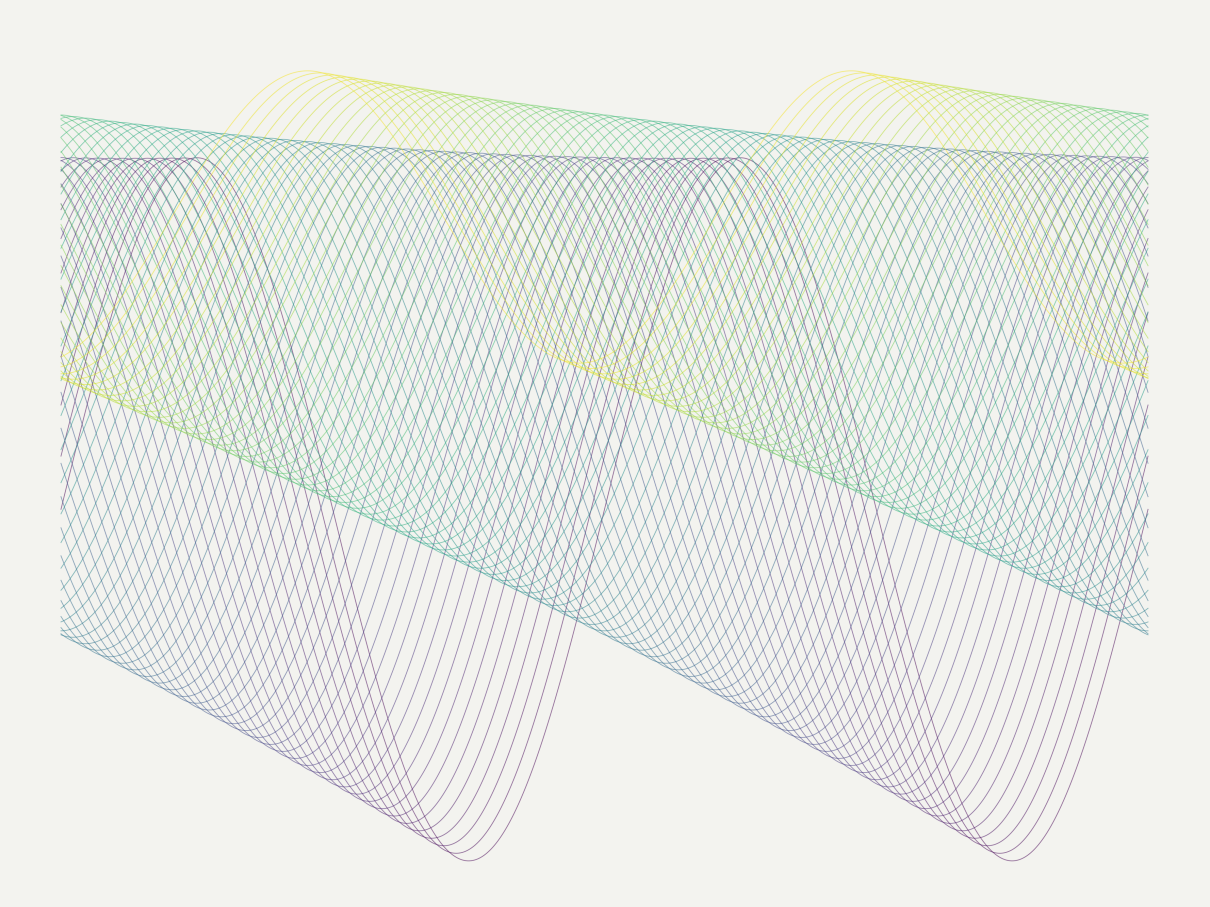


Figure 6: Technical visualization for project presentation

9. Conclusion

This project provides a practical Python-based approach to phishing URL detection. It combines URL feature extraction, machine learning, visual analysis, and optional web deployment. With regular dataset updates and additional domain intelligence, the system can help protect users from credential-stealing websites.